

DICKSON SARPONG

dicksonsarpong9@gmail.com | +233 555 732755 | Kumasi, Ghana | [GitHub](#) | [LinkedIn](#)

SUMMARY

AI Engineer specialized in Retrieval systems, Agentic systems, real-time voice agents and low-latency LLM systems. Proven track record in reducing p99 latency by 73% in my previous work and deploying autonomous agentic workflows using LangGraph, Groq API and Gemini Live API.

CORE SKILLS

- Voice & Audio: Gemini Live API, ElevenLabs, Deepgram, Whisper, TTS Fine-tuning, Automatic Speech Recognition, Livekit
- LLMs: Groq (Llama-3.3-70b), OpenAI APIs, Small Language models
- AI Systems: LLM agents, tool calling, prompt design, Model Context Protocol (MCP)
- ML infrastructure: Data processing, training pipelines, LoRA finetuning, Model versioning
- RAG: RAG architectures, hybrid search, FAISS, Qdrant, ChromaDB, vector search, BM25 ranking, cross-encoder reranking, similarity search
- Deployment: Docker, CI/CD (GitHub Actions), Railway, Modal, AWS(ECR, ECS, EC2)
- Monitoring: Prometheus, Grafana, LangSmith (LLM observability), OpenTelemetry
- Reliability: Circuit breakers, retry logic, graceful degradation, Safety guardrails
- Evaluation & Reliability: LangSmith, Hallucination tracking, Circuit breakers, RAG evaluation, Weight and Biases

EXPERIENCE

AI Engineer Intern | VoiceBreeze AI | Remote

November 2025 - February 2026

- Developed low-latency, two-way conversational agents using the Gemini Live API via LiveKit, implementing multimodal (Audio/Text) interactions
- Integrated real-time audio streams via WebRTC and LiveKit, ensuring sub-500ms processing for live voice-over-IP (VoIP) interactions
- Fine-tuned Nigerian-accent TTS models (VoxCPM, Spark-TTS 0.5B) on a 3.5-hour speech dataset segmented into 2,860 samples using Whisper, completing training in 6 hours on a Lambda H100 GPU.
- Built and deployed the production inference service on Modal with H100-backed infrastructure.
- Designed the entire TTS pipeline from raw audio to training, including normalization, VAD segmentation, Whisper transcription, and LoRA fine-tuning.
- Built FastAPI inference services for streaming and batch generation, voice cloning, and tunable latency vs quality settings depending on the request.
- Collaborated directly with the organization founder to design gateway APIs that reduced client integration complexity by 75% and added mixed precision and GPU warm-start strategies to avoid cold start penalties.
- Deployed multilingual speech-to-text for Hausa, Yoruba, and Igbo with request routing, retries, circuit breakers, and rate limiting so the service stays stable under load
- Implemented hybrid retrieval using semantic search and BM25 with reranking to improve

retrieval precision and reduce hallucinations

- Designed gateway API integrating AI services with external enterprise APIs and data systems, reducing client integration complexity from 4 endpoints to 1.
 - Tracked latency, request traces, and cost per request using LangSmith.
 - Deployed containerized ML services using Docker on AWS GPU infrastructure with autoscaling and streaming support
 - Integrated CI/CD pipelines for model versioning, deployment, and rollback
 - Authored internal technical documentation and reference implementations for FastAPI inference services to streamline AI adoption for software engineers.
 - Designed modular AI integrations using LangGraph and MCP, ensuring the system remains provider-agnostic and capable of swapping LLM backends (e.g., OpenAI, Groq) with minimal reconfiguration
-

PROJECTS

Hybrid Enterprise RAG System [GitHub](#) |

- Built hybrid RAG merging SQL analytics, time-series forecasting, and unstructured retrieval
- Implemented hybrid search (60% vector, 40% BM25) with cross-encoder re-ranking, improving Recall@10 from 0.78 to 0.92
- Configured circuit breakers, timeout thresholds, and graceful degradation routers to isolate component failures
- Built feedback APIs, failure dashboards, and active learning queues that route LLM errors to human-in-the-loop review
- Metrics: 100% SQL exact match, 11.02% forecast MAPE, 0.98 faithfulness, 1.5% hallucination rate
- Performance: p50 2.62s, p95 75.18s (ARIMA bottleneck identified), cost \$0.0018 per request
- Identified ARIMA sync training as P95 bottleneck; designed async solution (75s → <4s target)
- Packaged the services in Docker containers and configured a GitHub Actions CI/CD pipeline for automated testing and deployment

APRIS - Autonomous Public Repository Intelligence System [GitHub](#) |

- Built automated repository analysis using structured parsing before LLM reasoning
- Implemented sandboxed GitHub access through MCP with rate limits
- Implemented structured repository analysis using AST-inspired parsing techniques to extract functions, dependencies, and module relationships before LLM reasoning, improving context quality and reducing token usage
- Performance: <30s for 77-file analysis, cost ~\$0.05/repo, 86% confidence
- Built governance and safety mechanisms including RBAC enforcement, prompt-injection sanitization, token budgeting (100k max), and file size limits to ensure secure LLM interaction with untrusted code
- Observed LLM generation latency: p50 0.79s, p90 1.78s, p95 1.99s, p99 2.15s
- Average token consumption: ~934 tokens per request

VoxCPM: Production-Grade Nigerian Accent TTS Service [[Github](#)]

- Achieved ~10-20x real-time generation speed and <500ms first-byte latency for streaming audio by deploying a fine-tuned VoxCPM 1.5 model on H100 GPU infrastructure via Modal.
- Engineered a scalable FastAPI inference service supporting non-streaming, streaming, and batch processing with warm-start strategies to reduce container reuse latency to <1 second.
- Optimized operational costs by implementing a 5-minute idle scale-down window and volume caching, reducing monthly H100 overhead while maintaining high availability.
- Successfully adapted a 1.67 GB base model for Nigerian English using LoRA (Rank 64), targeting specific LM and DiT modules to ensure accent authenticity.
- Observability: Integrated Prometheus metrics and structured logging to track generation time, audio duration, and hardware utilization in real-time

SFT & DPO Fine-Tuning Pipeline | Python, PyTorch, Hugging Face [[GitHub link](#)]

- Built production-grade fine-tuning pipelines for LLM alignment using TinyLlama
 - Implemented Supervised Fine-Tuning (SFT) on instruction-response pairs
 - Implemented Direct Preference Optimization (DPO) for human alignment
 - Used LoRA and PEFT for efficient training (reduced params by 99%)
 - Tracked metrics: Training loss 0.69 → 0.56 (19% improvement)
 - Tech: Hugging Face Transformers, LoRA, PEFT, Weights & Biases
 - Merged adapters with base models for deployment-ready inference - Validated output quality through systematic testing
-

EDUCATION

Bachelor of Science in Statistics

Kwame Nkrumah University of Science and Technology

| August 2018 – November 2022

Kumasi-Ghana